Institut de sciences exactes
Département d'Informatique
Deuxième Année LMD-MIAS
Module : Algorithmes et structures de données

Solution de la fiche de TD n° 01 : complexité

Exercice 1 :

1. Réponse : 7.

Explication :

Supposons que vous ayez une liste triée de 128 noms et que vous utilisiez la recherche binaire. La recherche binaire divise la liste en deux à chaque étape, en vérifiant si l'élément que vous recherchez se trouve dans la moitié supérieure ou inférieure de la liste. Voici comment cela fonctionne :

- Étape 1 : On commence par le milieu de la liste, soit le nom au 64e emplacement. On compare le nom recherché avec celui-ci. S'il correspond, On a terminé. Sinon, on sait maintenant que le nom se trouve dans la première moitié ou la deuxième moitié de la liste.

- Étape 2 : Si le nom qu'on cherche est alphabétiquement inférieur au nom au 64e emplacement, on ignore la moitié supérieure de la liste. On est maintenant dans une liste de 64 noms. On répète le processus et on vérifie le nom au 32e emplacement.

- On continue à diviser la liste en deux à chaque étape, réduisant de moitié le nombre de noms à chaque fois. On répète ce processus jusqu'à ce qu'on trouve le nom qu'on recherche.

- Le maximum de 7 étapes est nécessaire car on divise la liste par 2 à chaque étape :

Étape 1	**Étape** 2	**Étape** 3	**Étape** 4	**Étape** 5	**Étape** 6	**Étape** 7
de 128 à 64	De 64 à 32	De 32 à 16	De 16 à 8	De 8 à 4	De 4 à 2	De 2 à 1

jusqu'à ce qu'on trouve le nom

2. Supposons qu'on double la taille de la liste. Quel est désormais le nombre maximum d'étapes ?

Réponse : 8.

Explication :

Si on double la taille de la liste à 256 noms, le nombre maximum d'étapes nécessaires dans le pire des cas augmentera légèrement. Voici pourquoi :

- Étape 1 : On commence au milieu de la liste, soit le nom au 128e emplacement (maintenant, à mi-chemin de la liste de 256 noms).

- Étape 2 : Si le nom qu'on cherche est alphabétiquement inférieur au nom au 128e emplacement, on ignore la moitié supérieure de la liste, ce qui nous laisse avec une liste de 128 noms.

- On continue à diviser la liste en deux à chaque étape, de la même manière que précédemment, jusqu'à ce qu'on trouve le nom.

- Le maximum de 8 étapes est nécessaire car nous divisons la liste par 2 à chaque étape (256 -> 128 -> 64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1) jusqu'à ce qu'on trouve le nom.

En résumé, lorsqu'on double la taille de la liste, la recherche binaire nécessitera une étape de plus (passant de 7 à 8 étapes) pour trouver un nom donné, car il y a plus d'éléments à parcourir, mais le principe de division par 2 à chaque étape reste le même.

Exercice n° 2 :

Voici un ensemble de problèmes traitant des données. Définissez la taille des données de l'entrée.

Problème	Taille de la donnée en entrée
Traverser dans un tableau	Le nombre d'éléments dans un tableau
Multiplier deux matrices	Les dimensions des matrices
Trier un tableau	Le nombre d'éléments dans un tableau
Traverser un arbinaire	Le nombre de nœuds
Résoudre un système d'équations linéaires	Le nombre d'équation, ou le nombre d'inconnues, ou les deux

Exercice 3 : les opérations fondamentales sont les suivant :

- Produit de deux variables de type entier : l'unique multiplication entre les deux nombres
- Recherche d'un élément dans une liste en mémoirecentrale : le nombre de comparaisons entre cet élément et les entrées de la liste
- Recherche d'un élément sur un disque : le nombre d'accès à la mémoire secondaire
- Tri d'une liste d'éléments : on peut considérer deux opérations fondamentales : le nombre de comparaisons entre deux éléments et le nombre de déplacements d'éléments.
- Multiplication de deux matrices de nombres : le nombre de multiplications et le nombre d'additions

Exercice n°4 :

Le programme de Fibonacci est :

```
procedure Fibonacci (n)
1.    read (n);
2.    if (n<0) then
3.        print ("error");
4.        exit ();
5.    if (n=0) then
6.    print ("Fibonacci number is 0") ;
7.    exit () :
8.    if (n=1) then
9.      print ("Fibonacci number is 1") ;
10.     exit ();
11.   fl=0;
12.   f2=1;
13.   for i = 2 to n do
14.   f = f1+ f2;
15.   f1 = f2 ;
16.   f2=f;
17.   end
18.   print ("Fibonacci number is", f) ;
end Fibonacci
```

Pour analyser le comportement du programme Fibonacci en termes du nombre d'opérations pour différentes valeurs de n, nous pouvons diviser les cas de n en trois catégories : n < 0, n = 0, n = 1, et n > 1.
Voici le tableau représentant le nombre d'opérations pour chaque ligne du code du programme :

Numéro de la ligne	Nombre d'opérations élémentaires			
	$n < 0$	$n = 0$	$n = 1$	$n > 1$
1	1	1	1	1
2	1	1	1	1
3, 4	1, 1	0	0	0
5	0	1	1	1
6, 7	0	1, 1	0	0
8	0	0	1	1
9, 10	0	0	1, 1	0
11, 12	0	0	0	1, 1
13	0	0	0	$(n-2+1)+1$
14, 15, 16	0	0	0	$(n-1), (n-1), (n-1)$
17	0	0	0	$(n-1)$
18	0	0	0	1
Total	**4**	**5**	**6**	$\mathbf{5n+3}$

Remarque : concernant la ligne 13 on a (n-2+1)+1 opérations car on a (n-2+1) pour l'itération et **+1** pour le test qui s'effectue à chaque fois.

Le nombre total de fréquences est de **5n + 3** et, par conséquent, la complexité temporelle du programme est T(n) = O(n).

```
def fibonacci(n):
    i = 1
    # du prochain test
    i += 1
    if n < 0:
        print("")
        i += 2
        return i
    # du prochain test
    i += 1
    if n == 0:
        print("")
        i += 2
        return i
    # du prochain test
    i += 1
    if n == 1:
        print("")
        i += 2
        return i
    f1, f2 = 0, 1
    i += 4
    for k in range(2, n + 1):
        f = f1 + f2
        f1 = f2
        f2 = f
        i += 5
    return i
# Initialisation du nombre d'opérations i à 0
i = 0
n = int(input("Entrez n pour calculer son nombre de Fibonacci: "))
# Appeler la fonction avec n en argument
j = fibonacci(n)
print("Le nombre Fibonacci est", j)
```

On a insérer dans le programme original le comptage du nombre d'opération .Là où il y a une opération élémentaire on fait un plus '+ ' de 1 sur la variable i.
Lancer Visual Studio et lancer l'exécution pour voir le nombre d'opération de ce programme.

<u>Exercice 5</u> : répondez aux questions suivantes : solution : livre algorithme en C de Kyle Loudon page 45

- **<u>Réponse 1</u>** : Il est évident que $O(1) \subset O(n) \subset O(n^2) \subset O(n^3)$, puisque :

$1 \leq n \leq n^2 \leq n^3 \; \forall n \geq 1$

On a également : $O(1) \subset O(\log n) \subset O(n) \subset O(n\log n)$, puisque :

$1 \leq \log n \quad \forall n \geq 3$ et $\log n \leq n \quad \forall n \geq 1$ et même pour la même raison, $O(n \log n) \subset O(n^2)$

On montre facilement que $O(n^3) \subset O(2^n)$.

En effet, $n^3 \leq 2^n \forall\ n \geq 10 \Leftrightarrow \text{Log } n^3 \leq \log 2^n \Leftrightarrow 3 \log n \leq n \log(2)$

Qui est vrai à partir d'une certaine valeur de n puisque $\log n \in O(n)$.

L'ordre d'inclusion est donc finalement :

$\mathbf{O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^2 \log n) \subset O(n^3) \subset O(2^n) \subset O(3^n) \subset O(n!)}$**,**

Un algorithme ayant une complexité logarithmique sera donc meilleur (en terme de complexité) qu'un algorithme linéaire, lui-même meilleur qu'un algorithme polynomiale, à son tour meilleur qu'un algorithme exponentiel.

- **<u>Réponse 2</u>** : Selon les règles de la notation O, les complexités de $T_1(n)$, T2(n) et T3(n) sont, respectivement, $O(n \log n)$, $O(2^n)$ et $O(n)$. On a, dans l'ordre croissant : $O(n)$, $O(n \log n)$ et $O(2^n)$.
- **<u>Réponse3 :</u>**Pour additionner m matrices de n x n, il faut effectuer m – 1 additions nécessitant chacune un temps de $O(n^2)$. Par conséquent, le temps d'exécution global est : $O(m-1)O(n^2)=O(m)O(n^2)=O(mn^2)$

- **Réponse 4 :** La complexité de T1 est O(n), et celle de T2 est $O(n^2)$. Cependant, dans l'algorithme décrit par T1, le coefficient constant de n est si grand que, lorsque n<400, celui décrit par T2 est préférable. Il s'agit d'un bon exemple de l'importance d'autres facteurs que la seule complexité d'un algorithme.

Exercice 6 : Le tableau suivant est calculé en utilisant un logarithme népérien :

f(n)	10	20	30	60
Log n	$2{,}30 \times 10^{-6}$	$3{,}00 \times 10^{-6}$	$3{,}40 \times 10^{-6}$	$4{,}09 \times 10^{-6}$
n	10^{-5}	2×10^{-5}	3×10^{-5}	6×10^{-5}
n log n	$2{,}30 \times 10^{-5}$	$5{,}99 \times 10^{-5}$	$1{,}02 \times 10^{-4}$	$2{,}46 \times 10^{-4}$
$\mathbf{n^2}$	10^{-4}	4×10^{-4}	9×10^{-4}	$3{,}60 \times 10^{-3}$
$\mathbf{n^3}$	10^{-3}	8×10^{-3}	$2{,}70 \times 10^{-2}$	$2{,}16 \times 10^{-1}$
$\mathbf{2^n}$	$1{,}02 \times 10^{-3}$	1,05	$1{,}07 \times 10^{3}$	$1{,}15 \times 10^{12}$

Il est nécessaire de remarquer que $1{,}15 \times 10^{12}$s $\approx$ 36558 ans

Exercice 7:

1- Si $f(n) \in O(g(n))$, alors il existe $k>0$ et $n_0 \geq 0$ tels que : $f(n) \leq kg(n)\ \forall n \geq n_0$

On déduit de l'expression précédente que :

$f(n)+g(n) \leq (k+1)g(n)\ \forall n \geq n_0$

Donc f(n) + g(n) est en O(g(n))

2- Pour montrer l'égalité des deux ensembles O(f(n)+g(n)) et O(max(f(n),g(n))), il faut montrer la double inclusion

$O(f(n)+g(n)) \subset O(max(f(n),g(n)))$:

$O(max(f(n),g(n))) \subset O(f(n)+g(n))$

1- $O(f(n)+g(n)) \subset O(max(f(n),g(n)))$:

Pour toute fonction $h(n) \in O(f(n)+g(n))$, il existe $k>0$ et $n0 \geq 0$ tels que :

$h(n) \leq k\,(f(n) + g(n)) \leq 2\,k \max(f(n),g(n))\ \forall n \geq n_0$

Donc h(n) est donc en O(max (f(n)+g(n)))

2- $O(max(f(n),g(n))) \subset O(f(n)+g(n))$

Pour toute fonction $h(n) \in O(max(f(n),g(n))$, il existe $k>0$ et $n0 \geq 0$ tels que :

$h(n) \leq k \max(f(n), g(n)) \leq k(f(n)+g(n))\ \forall n \geq n_0$

Donc h(n) est en O(f(n)+g(n))

Exercice n° 08:

1. Le programme calculant le produit des deux matrices A et B est le suivant :
   - **En langage Pascal**

```
Program Produit_matrice;
    Var n,i,j,k:Integer;
    A,B,C:array[1..10,1..10] of Integer;
Begin
    Write('La dimension <10:');
    Read(n);
    Write('Lecture des matrices');
    For i:=1 To N Do
    For j:=1 To N Do
        Read(A[i,j],B[i,j]);
    Write('Produit des deux matrices A et B');
    For i:=1 To N Do
        For j:=1 To N Do
        Begin
                C[i,j):=0;
                For k:=1 To N Do
                        C[i,j]:= C[i,j]+A[i,k]*B[k,j];
        End;
    Write('Lecture des matrices');
    For i:=1 To N Do
    For j:=1 To N Do
        Write(C[i,j]);
End.
```

- **En langage Python :**

```
# Déclaration des variables
n = int(input("La dimension <10: "))
A = [[0] * 10 for _ in range(10)]
B = [[0] * 10 for _ in range(10)]
C = [[0] * 10 for _ in range(10)]

# Lecture des matrices A et B
print("Lecture des matrices A et B:")
for i in range(1, n + 1):
    for j in range(1, n + 1):
        A[i][j] = int(input(f"A[{i},{j}]: "))
        B[i][j] = int(input(f"B[{i},{j}]: "))

# Calcul du produit des matrices A et B
print("Produit des deux matrices A et B:")
for i in range(1, n + 1):
    for j in range(1, n + 1):
        C[i][j] = 0
        for k in range(1, n + 1):
            C[i][j] += A[i][k] * B[k][j]

# Affichage de la matrice résultante C
print("Matrice résultante C:")
for i in range(1, n + 1):
    for j in range(1, n + 1):
        print(C[i][j], end=" ")
    print()
```

Le nombre d'opérations effectuées par le programme :
$2n^2$ pour la lecture des données des deux matrices
n^2 pour l'affichage de la matrice résultat C
n^3 pour le calcul de la matrice produit C
Donc on a : n^3+3n^2
On en déduit que le programme est en $O(n^3)$

2. Le programme modifié est le suivant :
   - En Pascal :

```
Program Produit_matrice;
    Varm,n,p,i,j,k:Integer;
    A,B,C:array[1..10,1..10] of Integer;
Begin
    Write('La dimension <10:');
    Read(n);
    Write('Lecture des matrices');
    For i:=1 To M Do
    For j:=1 To N Do
        Read(A[i,j]);
    For i:=1 To N Do
    For j:=1 To P Do
        Read(B[i,j]);
    Write('Produit des deux matrices A et B');
    For i:=1 To M Do
```

```
                For j:=1 To P Do
                Begin
                        C[i,j):=0;
                        For k:=1 To N Do
                                C[i,j]:= C[i,j]+A[i,k]*B[k,j];
                End;
            Write('Lecture des matrices');
            For i:=1 To M Do
            For j:=1 To P Do
                Write(C[I,j]);
End.
```

- En Python

```
# Déclaration des variables
m = int(input("Nombre de lignes de la matrice A: "))
n = int(input("Nombre de colonnes de la matrice A: "))
p = int(input("Nombre de colonnes de la matrice B: "))
A = [[0] * n for _ in range(m)]
B = [[0] * p for _ in range(n)]
C = [[0] * p for _ in range(m)]
# Lecture des matrices A et B
print("Lecture de la matrice A:")
for i in range(m):
    for j in range(n):
        A[i][j] = int(input(f"A[{i + 1},{j + 1}]: "))
print("Lecture de la matrice B:")
for i in range(n):
    for j in range(p):
        B[i][j] = int(input(f"B[{i + 1},{j + 1}]: "))
# Calcul du produit des matrices A et B
print("Produit des deux matrices A et B:")
for i in range(m):
    for j in range(p):
        C[i][j] = 0
        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
# Affichage de la matrice résultante C
print("Matrice résultante C:")
for i in range(m):
    for j in range(p):
        print(C[i][j], end=" ")
    print()
```

Le nombred'opérations effectuées par le programme :
mn+np pour la lecture des données des deux matrices
mp pour l'affichage de la matrice résultat C
mnp pour le calcul de la matrice produit C
Donc on a : mn+np+mp+mnp
On en déduit que le programme est en O(mnp)

<u>Exercice n° 09 :</u>

- En pascal :

```
Program Somme_tableau;
    VarS,n,i:Integer;
    A:array[1..10] of Integer;
Begin
```

```
            Write('La dimension <10:');
            Read(n);
            Write('Lecture du tableau');
            For i:=1 To N Do
                Read(A[i]);
            S:=0;
            For i:=1 To N Do
                S:=S+A[i];
            Write('La somme des éléments du tableau est :',S);
End.
```

- **En Python :**

```
# Déclaration des variables
n = int(input("La dimension <10: "))
A = [0] * 10  # Crée un tableau de 10 éléments initialisés à zéro

# Lecture du tableau
print("Lecture du tableau:")
for i in range(1, n + 1):
    A[i] = int(input(f"A[{i}]: "))

# Calcul de la somme des éléments du tableau
S = 0
for i in range(1, n + 1):
    S += A[i]

# Affichage de la somme
print("La somme des éléments du tableau est :", S)
```

Pour prouver ce programme il faut démontrer deux choses : la terminaison et la validité du programme.
La terminaison du programme est triviale puisque les deux boucles sont exécutées chacune n fois et que le corps des deux boucles n'est constitué premièrement par une lecture et deuxièmement par une somme et une affectation, opérations qui s'exécutent en un temps fini.

Pour démontrer la validité, nous allons considérer seulement la boucle suivante :

En Pascal	En Python
S:=0; **For i:=1 To N Do** **S:=S+A[i];**	S = 0 for i in range(1, n + 1): S += A[i]

On a la propriété suivante :
"à la fin de l'itération i, la variable S contient des i premiers éléments du tableau A"
Notons S_i la valeur de la variable S à la fin de l'itération i et $S_0=0$ sa valeur initiale. On effectue alors un raisonnement par récurrence sur i. La propriété est trivialement vraie pour i=1 puisqu'à la fin de la première itération S (initialisée à 0) contient la valeur A[1] :
$S_1=S_0 + A[1] = A[1]$
Maintenant, si on suppose que la propriété est vraie pour une itération i :

$$S_i=\sum_{j=1}^{i} A[j]$$

A la fin de l'itération i+1, S contiendra la valeur qu'elle contenait à la fin de l'itération i, donc la somme des i premiers éléments, plus A[i+1] qui lui a été ajouté à l'itération i+1. S sera donc bien la somme des i+1 premiers éléments de A :

$$Si+1 = Si + A[i+1] = \sum_{j=1}^{i} A[j] + A[i+1] = \sum_{j=1}^{i+1} A[j]$$

Vraie initialement, la propriété reste à chaque nouvelle itération. Le programme se termine, à la fin de l'itération n, il aura donc bien calculé la somme des n éléments du tableau A.

2. Le nombre d'opérations effectuées par le programme :
n lectures et n additions. Donc le programme est en O(n)

3. Le programme récursif calculant la somme des éléments du tableau A est la suivante :
Le programme récursif est un programme qui fait appel à des sous programmes récursifs.
Dans notre cas, ce sera une fonction récursive qui réalise la somme des éléments du tableau A.

La fonction récursive est :

En Pascal	En Python

Function Somme(A:array[1..10] of Integer,n:integer):Integer;
Begin
If n<=0 then Somme:=0
Else Somme :=Somme(A,n-1)+A[n];
End;
	def Somme(A, n):
	  if n <= 0:
	  return 0
	  else:
	  return Somme(A, n - 1) + A[n - 1]

On démontre tout d'abord la terminaison, par récurrence sur i. de façon évidente l'appel de la fonction Somme avec un paramètre 0 se termine immédiatement. Si on suppose que l'appel de la fonction Somme avec un paramètre k se termine, l'appel de la fonction Somme avec le paramètre k+1, appelle la fonction Somme avec le paramètre k, qui par hypothèse se termine, ajoute à son résultat la valeur A[k] et se termine. Donc quelle que soit la valeur du paramètre k, l'appel de la fonction Somme se termine.
On note $Somme_n$ la valeur retournée par l'appel de la fonction Somme avec le paramètre n. cette fonction récursive reproduit la relation mathématique de récurrence suivante :

$$\begin{cases} Somme_0 = 0 \\ Somme_n = Somme_{n-1} + A[n] \quad si\ n > 0 \end{cases}$$

On montre facilement par récurrence (exactement de la même façon que cela a été fait dans la question précédente), la solution de cette expression récursive est :

$$Somme_n = \sum_{i=1}^{n} A[i]$$

Dès l'instant où la fonction se termine (ce qu'on a prouvé), elle calcule bien la somme des éléments du tableau.
4. On appelle a(n) le nombre d'additions effectuées par la fonction Somme appelée avec un paramètre n. a(n) est solution de l'équation de récurrence :

$$a(n) = \begin{cases} 0 \quad si\ n = 0 \\ a(n-1) + 1 \quad si\ n > 0 \end{cases}$$

Dont la solution est a(n)=n. on en déduit que l'algorithme est en O(n).

<u>Exercice n° 10:</u>

Le corps des différents programmes ne contient que des opérations en O(1). En comptant le nombre d'itérations effectuées par chacun des algorithmes, on obtient que :

- le programme A est en O(min(m,n))
- le programme B est en O(max(m,n))
- le programme C est en O(m+n)
- le programme D est en O(mn)

<u>Exercice n° 11 :</u>

1- On calcule les premieres valeurs calculées par la function F :

$F(0)=2$
$F(1)=2*2=2^2$
$F(2)=2^2*2^2=2^4$
$F(3)=2^4*2^4=2^8$
$F(4)=2^8*2^8=2^{16}$

De façon intuitive, on obtient l'expression générale de F(n) :

F(n)= 2^{2^n}

2- On s'intéresse au nombre m(n) de multiplications effectuées. M(n) est solution de l'équation de récurrence :

$$m(n) = \begin{cases} 0 & si \;\; n = 0 \\ 1 + 2 * m(n-1) & si \;\; n > 0 \end{cases}$$

On montre très facilement par récurrence que m(n)= $\sum_{i=0}^{n-1} 2^i = 2^n - 1$. On en déduit que l'algorithme est en O(2^n) : complexité expenentielle.

Pour améliorer l'algorithme, il suffit bien évidemment d'éviter le double appel récursif; la fonction F s'écrit :

En Pascal	En Python

Function F(n:Integer):Integer;
Begin
  If n=0 then F:=2
  Else F:=F(SQR(n-1))
End;
	def F(n):
	  if n == 0:
	  return 2
	  else:
	  return F((n - 1) ** 2)

On calcule alors la complexité par rapport au nombre c(n) d'appels de la fonction "carré" . c(n) est solution de l'équation de récurrence :

$$c(n) = \begin{cases} 0 & si \;\; n = 0 \\ c(n-1) + 1 & si \;\; n > 0 \end{cases}$$

De façon évidente, c(n)=n. on en déduit que le nouvel algorithme est en O(n).

2- La fonction G calcule la même valeur que la fonction F. si m(n) est à nouveau le nombre de multiplication effectuées par l'algorithme, de façon évidente, m(n)=n. on en déduit que l'algorithme est en O(n).